Walkthrough: Build the “Explore the Outdoors” Parallax Page

You will build a nature page where two photo banners stay in place while the text sections scroll over them. Each step shows you the HTML and CSS for one section of the page, explains what every part does, and tells you why that code is a good choice.

Before You Start

- **You need:** a code editor (such as VS Code), a web browser, and two landscape photos One should show mountains and the other the ocean or a coastline.
- **You should know:** basic HTML tags and how to link a CSS file. You will learn the Flexbox parts as you go.

How to Use This Walkthrough

Each step has four parts: **code to type**, **what it does**, **why it helps**, and a **checkpoint**.
Follow these rules:

1. Type the code yourself instead of pasting it.
2. Save both files and refresh your browser after each step.

What You Are Building

Page section	HTML element	CSS class	Flexbox used for
Top banner with menu	`header`	`.hero`, `.navbar`, `.heroContent`	Stacking the menu above the title, and centering both
Text sections (3)	`section`	`.contentSection`	Not needed: normal flow with a centered column
Photo banners (2)	`section`	`.parallax` plus `.mountains` or `.ocean`	Centering a heading on top of a photo
Footer	`footer`	`.footer`	Not needed: simple centered text

Step 1: Set Up Your Files

Create a folder where everything lives.

1. Create a folder named `outdoors-page`.
2. Inside it, create `index.html` and `styles.css`.
3. Save your two photos in the same folder. Name them `mountains.webp` and `oceans.jpg`, or use your own names and change them in Step 5.

Why it helps: keeping the HTML, CSS, and images together means every file path is short and easy to get right. Most broken-image problems come from a wrong path or file name.

Step 2: Build the Page Foundation

Goal: write the HTML boilerplate and the base styles every other rule depends on.

In `index.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Explore the Outdoors</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>

    <!-- Steps 3 to 7 go here -->

</body>
</html>
```

In `styles.css`:

```
* {
    box-sizing: border-box;
}

body {
    margin: 0;
    font-family: Arial, sans-serif;
    line-height: 1.6;
    color: #26332e;
}
```

What it does:

- `lang="en"` tells the browser and screen readers that the page is in English.
- The `viewport` meta tag makes the page scale to the width of a phone screen.
- `box-sizing: border-box` makes an element's padding and border count inside its width.
- `margin: 0` removes the default gap the browser puts around the page.
- `line-height: 1.6` adds space between lines of text.

Why it helps:

- `box-sizing: border-box` makes sizes predictable. A box set to `max-width: 900px` stays 900 pixels wide even after you add padding, so your layout does not break unexpectedly.
- Removing the body margin lets the colored header and the photo banners reach the edges of the window.
- A line height of 1.6 makes paragraphs easier to read.
- The dark green-gray text color is softer than pure black but still has strong contrast against white.

Checkpoint: the browser tab shows "Explore the Outdoors" and the page is blank.

Step 3: Create the Hero Header and Navigation

Build the top banner with a menu bar and a centered title.

In `index.html`, replace the comment with:

```
        <header class="hero" id="home">
            <nav class="navbar" aria-label="Main navigation">
                <a href="#home">Home</a>
            </nav>

            <div class="heroContent">
                <h1>Explore the Outdoors</h1>
                <p>Discover the beauty of the natural world.</p>
            </div>
        </header>

        <!-- Steps 4 to 7 go here -->
```

In `styles.css`, add:

```
    .hero {
        min-height: 50vh;
        display: flex;
        flex-direction: column;
        color: white;
        background-color: #294c60;
    }

    .navbar {
        display: flex;
        justify-content: center;
        align-items: center;
        gap: 10px;
        padding: 10px;
        background-color: #173b32;
    }

    .navbar a {
        color: white;
        text-decoration: none;
        font-weight: bold;
    }

    .navbar a:hover {
        text-decoration: underline;
    }

    .heroContent {
        flex: 1;
        display: flex;
        flex-direction: column;
        justify-content: center;
        align-items: center;
        text-align: center;
        padding: 10px;
    }

    .heroContent h1 {
        font-size: 2rem;
        line-height: 1.2;
        margin: 10px;
    }

    .heroContent p {
        font-size: 1.2rem;
        margin: 0;
    }
```

What it does:

- `.hero` is a flex container with `flex-direction: column`, so the navbar sits on top and the title area sits below it.
- `min-height: 50vh` makes the header at least half the height of the browser window.
- `.navbar` is a flex container too. `justify-content: center` centers the links across the bar, `align-items: center` centers them vertically, and `gap: 10px` spaces them apart.
- `flex: 1` on `.heroContent` tells it to grow and fill all the space left under the navbar.
- `.heroContent` is also a flex container with a column direction, so `justify-content: center` centers the title vertically and `align-items: center` centers it horizontally.

Why it helps:

- Using `min-height` instead of `height` means the header grows if the text gets longer, so nothing is cut off.
- `flex: 1` pins the menu to the top while the title stays centered in the remaining space, with no manual margins or pixel math.
- Nesting one flex container inside another is a common pattern, so you will use it again in real projects.
- The `aria-label` gives screen reader users a name for the menu. The `id="home"` gives the Home link something to jump to.
- The background color is a backup that keeps the white text readable if you later add an image.

Checkpoint: you see a dark blue header with a green menu bar at the top and a centered title and tagline below it. Clicking Home keeps you at the top.

Step 4: Add the First Content Section

Create a readable text section in a centered column. In

`index.html`, replace the comment with:

```
            <main>
                <section class="contentSection">
                    <h2 id="adventureTitle">Adventure Awaits</h2>
                    <p>
                        The outdoors offers opportunities to explore,
                        learn, and experience the beauty of nature.
                        From towering mountains to peaceful coastlines,
                        there is always something new to discover.
                    </p>
                </section>

                <!-- Steps 5 and 6 go here -->
            </main>

            <!-- Step 7 goes here -->
```

In `styles.css`, add:

```
        .contentSection {
            max-width: 900px;
            margin: 0 auto;
            padding: 60px 20px;
            text-align: center;
        }

        .contentSection h2 {
            color: #245b4b;
        }
```

What it does:

- `max-width: 900px` stops the section from getting wider than 900 pixels.
- `margin: 0 auto` puts equal automatic space on the left and right, which centers the section.
- `padding: 60px 20px` adds breathing room above and below the text and keeps it away from the screen edges on small devices.
- `.contentSection h2` is a descendant selector, so only headings inside a content section turn green.

Why it helps:

- Long lines of text are tiring to read. Capping the width keeps lines at a comfortable length on wide monitors.
- `max-width` (instead of `width`) lets the section shrink on phones.
- Putting the style on a class instead of the `section` tag means you can reuse it for every text section with no new CSS.

- Using `main` for the page's primary content helps search engines and screen readers find it.

Checkpoint: a centered heading and paragraph appear under the header, and the text never stretches across a wide window.

Step 5: Add Your First Parallax Banner

Create a photo banner that stays still while the page scrolls over it. In

`index.html`, replace the Steps 5 and 6 comment with:

```
            <section class="parallax mountains">
                <h2>Find Your Summit</h2>
            </section>

            <!-- Step 6 goes here -->
```

In `styles.css`, add:

```
        .parallax {
            min-height: 400px;
            display: flex;
            justify-content: center;
            align-items: center;
            text-align: center;
            color: white;
            background-size: cover;
            background-position: center;
            background-repeat: no-repeat;
            background-attachment: fixed;
        }

        .parallax h2 {
            max-width: 90%;
            margin: 20px;
            padding: 20px;
            font-size: 2.5rem;
            line-height: 1.2;
            background-color: #000000;
        }

        .mountains {
            background-image: url("mountains.webp");
        }
```

What it does:

- `display: flex` with `justify-content: center` and `align-items: center` puts the heading in the exact middle of the banner.
- `background-size: cover` scales the photo to fill the whole section without stretching it.
- `background-position: center` keeps the middle of the photo visible when it is cropped.
- `background-repeat: no-repeat` stops the photo from tiling.
- `background-attachment: fixed` is the parallax effect. It pins the photo to the window, so the banner acts like a window that slides over the picture as you scroll.
- `.parallax h2` gives the heading a solid black box so white text is readable on any photo.

Why it helps:

- Splitting the code into `.parallax` (the shared look) and `.mountains` (just the picture) means the shared rules are written once. Adding another banner later only takes one new class.
- `min-height: 400px` lets the banner grow with its content instead of clipping it.
- `max-width: 90%` on the heading keeps it from touching the screen edges on narrow screens.
- A solid background behind text keeps your page readable regardless of which photo you choose.

- This effect uses only CSS, with no JavaScript, so it loads fast and is simple to maintain.

Heads up: many phone browsers, including Safari on iPhone, ignore `background-attachment: fixed`. On those devices the photo simply scrolls with the page. The page still works, but the effect is not visible.

Checkpoint: you see a photo banner with a centered heading in a black box. When you scroll, the photo stays still while the banner edges slide over it.

Step 6: Repeat the Pattern for the Rest of the Page

Finish the remaining three sections by reusing the classes you already wrote. In

`index.html`, replace the Step 6 comment with:

```
            <section class="contentSection">
                <h2>Explore the Mountains</h2>
                <p>
                    Mountains offer breathtaking views, challenging
                    trails, and opportunities to appreciate the
                    natural world. Every journey provides a new
                    perspective.
                </p>
            </section>

            <section class="parallax ocean">
                <h2>Discover the Coast</h2>
            </section>

            <section class="contentSection">
                <h2>Enjoy the Ocean</h2>
                <p>
                    The ocean is home to incredible wildlife,
                    beautiful coastlines, and constantly changing
                    landscapes. Protecting these environments helps
                    preserve them for future generations.
                </p>
            </section>
```

In `styles.css`, add one new rule under `.mountains`:

```
    .ocean {
        background-image: url("oceans.jpg");
    }
```

Why it helps:

- The two new content sections and the new banner needed **no new layout CSS**. You only added one line for the new picture. This is the main benefit of reusable classes: more content, less code.
- Alternating text sections and photo banners gives the reader a rhythm and breaks a long page into easy chunks.
- To change a photo later, you edit one line in one place.

Checkpoint: your page now alternates between text and photo banners (text, mountains, text, ocean, text). Both photos stay fixed as you scroll.

Step 7: Add the Footer

Close the page with a footer that matches your header. In

`index.html`, replace the Step 7 comment with:

```
        <footer class="footer">
            <p>Explore the Outdoors | Your Name</p>
        </footer>
```

In `styles.css`, add:

```
        .footer {
            padding: 25px;
            text-align: center;
            background-color: #173b32;
            color: white;
        }

        .footer p {
            margin: 0;
        }
```

What it does:

- `padding: 25px` gives the footer height without setting a fixed height.
- The background color is the same green as the navbar, so the top and bottom of the page match.
- `margin: 0` on the paragraph removes the default space under the text, which would otherwise add extra green at the bottom.

Why it helps: reusing the navbar's color ties the design together. Using the `footer` tag instead of a plain `div` tells browsers and screen readers that this is the end-of-page information.

Checkpoint: a dark green footer with white, centered text sits at the bottom of the page.

Step 8: Test and Troubleshoot

Scroll the whole page, then resize your browser window to a narrow width. Open your browser's developer tools (press F12) and use the device toolbar to preview a phone size.

Problem	Likely cause	Fix
A photo does not show	The file name or path in `url()` does not match the file, or the capitalization is different	Compare `url("...")` to the exact file name in your folder
The photo scrolls with the page	`background-attachment: fixed` is missing or misspelled, or you are on a phone	Check the `.parallax` rule. On a phone this is expected
The photo looks stretched or repeats	`background-size` or `background-repeat` is missing	Add `cover` and `no-repeat` to the `.parallax` rule
The heading is not centered on the photo	`display: flex`, `justify-content`, or `align-items` is missing	Check the `.parallax` rule
The page has a white gap at the top	`margin: 0` is missing on `body`	Add it to your `body` rule from Step 2
Styles do not appear at all	The CSS file name or `link` is wrong	Check that `href="styles.css"` matches your file name exactly
Changes do not show	The file was not saved, or the browser kept an old copy	Save, then hard refresh (Ctrl+Shift+R, or Cmd+Shift+R on a Mac)